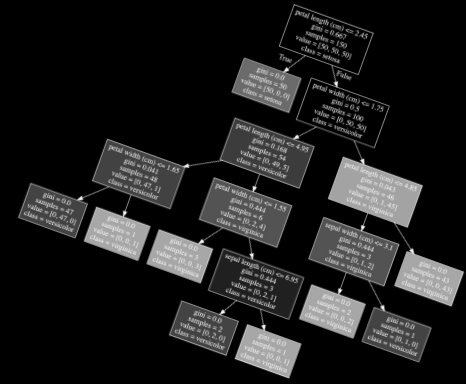
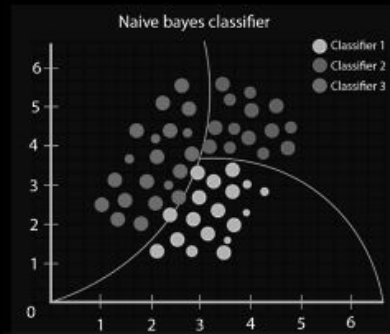
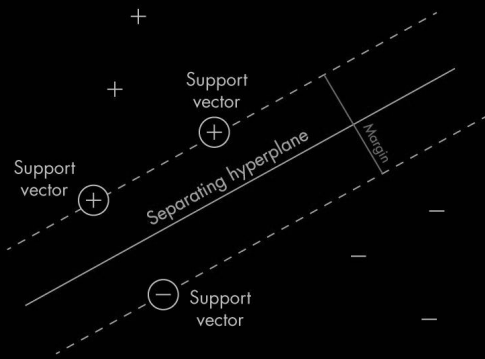




Data Science Crash Course

Objective: Mastering data structures, feature engineering, and non-neural classification algorithms before exploring deep learning architectures

What is Data? (The Raw Material)

Def: Data is any quantifiable representation of real-world phenomena

Numerical/Quantitative

- > Represents measurable quantities.
- > **Continuous:** Values that can be continuously subdivided (salary, temperature).
- > **Discrete:** Countable distinct values (number of students, CPU cores).

Categorical/Qualitative

- > Represents characteristics or groups.
- > **Nominal:** Labels with no inherent mathematical order (country, color).
- > **Ordinal:** Labels with a meaningful, sequential scale (pain ratings, Likert scale).

Features: Mapping Inputs to Outputs

Features (X): The independent variables, dimensions, or measured attributes used by an algorithm as predictors.

Target Variable (Y): The dependent variable, outcome, or label we want our machine learning model to predict.

Feature Engineering

> The art of selecting, transforming, and combining raw observations into descriptive columns.

> These columns expose the underlying patterns to the model.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

The last column
is the target
variable

Features for the Iris dataset

Preprocessing: Preparing Data for the Machine

Real world data is dirty: we must systematically handle missing/corrupt values, encode categorical text into numbers, and scale our feature so our machine can learn efficiently.

Missing Value Strategies

- > **Imputation:** Replacing blank fields with statistical substitutes like column mean/medians.
- > **Deletion:** Dropping the incomplete row or feature column entirely (Risk: dropping valuable data variance).

Categorical Encoding

- > **One-Hot Encoding:** Converts nominal categories into distinct binary vector columns (0 or 1) to eliminate arbitrary ordering bias.
- > **Label Encoding:** Assigns sequential integers to ordinal text values where an inherent ranking exists (small = 1, med = 2, large = 3).

Feature Scaling

- > **Standardization:** Centers data to have a mean of 0 and a std of 1 (important for SVM and geometric models).
- > **Normalization:** Bounds and compresses continuous values strictly within a uniform range, usually (0, 1).

The Evaluation Framework: Validation Protocols

The Train/Test Split Framework:

- > **The Partition:** Dividing your clean, processed dataset into two distinct, non-overlapping segments. Usually 80% for training and 20% for testing.
- > **Goal:** Preserving a completely unseen validation subset to simulate how the model will perform in the real world.



The Cross Validation Framework:

- > **Def:** A technique where you divide your dataset into smaller chunks, and then train the model on some chunks and test on others and repeating this process multiple times.
- > **Goal:** Ensures the model's performance metrics are highly stable and not merely a statistical artifact of a lucky initial random shuffle.

Titanic Dataset Preprocessing

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

raw_df = sns.load_dataset('titanic')
raw_df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

```
columns_to_keep = ['survived', 'pclass', 'sex', 'age', 'fare', 'embarked']
df = raw_df[columns_to_keep].copy()

print(f"Dataset Shape: {df.shape}")
df.head()
```

Dataset Shape: (891, 6)

	survived	pclass	sex	age	fare	embarked
0	0	3	male	22.0	7.2500	S
1	1	1	female	38.0	71.2833	C
2	1	3	female	26.0	7.9250	S
3	1	1	female	35.0	53.1000	S
4	0	3	male	35.0	8.0500	S

Goal: Predict if a person survived or perished

Titanic Dataset Preprocessing Cont.

```
# Check the number of null values
print(df.isnull().sum())

# 1. Missing Value Strats (Imputation vs. Deletion)
# We impute continuous 'age' with the median to avoid outlier bias
df['age'] = df['age'].fillna(df['age'].median())

# We drop rows with missing 'embarked' since it's a tiny fraction (only 2 rows)
df = df.dropna(subset=['embarked'])

# Check for null values again after preprocessing
print("----- After Preprocessing -----")
print(df.isnull().sum())
```

```
survived    0
pclass      0
sex         0
age        177
fare        0
embarked     2
dtype: int64
----- After Preprocessing -----
survived    0
pclass      0
sex         0
age         0
fare        0
embarked    0
dtype: int64
```

Titanic Dataset Preprocessing Cont.

```
from sklearn.model_selection import train_test_split, cross_val_score

# 2. Train/Test Split
X = df.drop(columns=['survived'])
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)
```


Titanic Dataset Preprocessing Cont.

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

# 3. Categorical Encoding & Feature Scaling
# Define which columns get what mathematical transformation
numeric_features = ['age', 'fare']
categorical_features = ['sex', 'embarked', 'pclass'] # pclass is treated as
categorical ordinal

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(drop='first', sparse_output=False),
         categorical_features)
    ]
)

# Fit on training data ONLY, transform both
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print(f"\nProcessed Training Feature Shape: {X_train_processed.shape}")
```

Processed Training Feature Shape: (711, 7)

Introduction to Model Training

Big Idea: Training is the process where an algorithm finds the mathematical patterns that map inputs to outputs.

Supervised Learning

Mechanism: Learning with a teacher. The data contains both the features (X) and the answer key (Y).

Goal: Predict the label for new, unseen data.

Main Tasks: Classification (is this a dog?), Regression (Predict the stock prices).

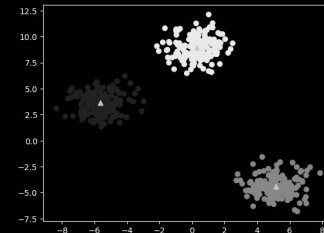


Unsupervised Learning

Mechanism: Learning independently. The data has features, but no label.

Goal: Discover hidden groupings or compress complex information.

Main Tasks: Clustering (grouping customers by behavior), Association (People who buy bread also buy butter).



Teaching Machines to Categorize

Classification: A type of Supervised Learning where the goal is to predict a discrete label or category.

Binary Classification



Spam

vs



Not Spam



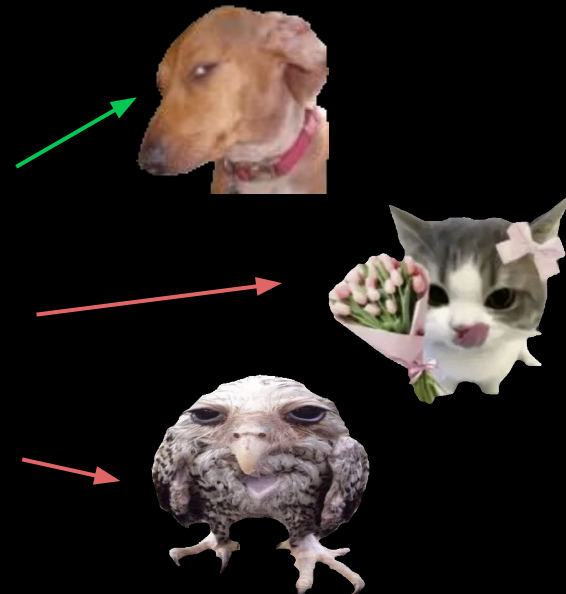
Bad Movie

vs



Good Movie

Multiclass Classification



Naive Bayes: The Probability Approach

The Big Idea: Evaluates data by calculating the conditional probability of the target class based on the vector of features present in the input.

The Formula:

$$P(Y|X) = [P(X|Y) * P(Y)] / P(X)$$

Why Naive?

It relies on a strict mathematical assumption that all input features are entirely independent of one another given the class label.



Incoming email

Naive Bayes:
 $P(\text{SPAM} \mid \text{TEXT FEATURES}) = 95\%$



Classify as SPAM

Decision Trees: Logic-Based Learning

Big Idea: An algorithmic “Choose Your Own Adventure” game.

Core Mechanics:

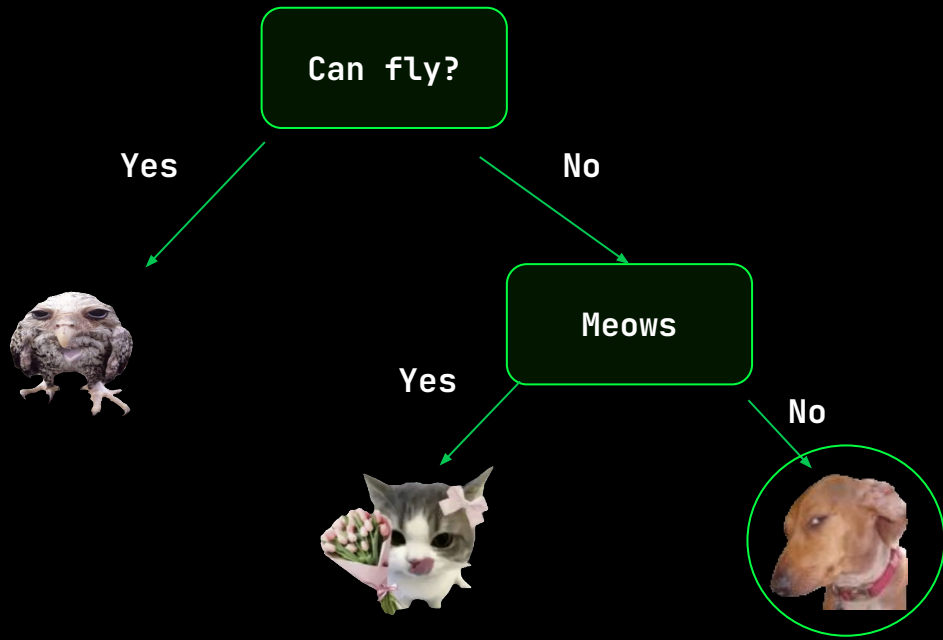
> **Recursive Splitting:** Model asks a series of sequential if/then questions based on data features

> **Interpretability:** Every decision path can be visually traced.

> **The Overfitting Trap:** Sometimes can create a tree so deep it fails to generalize to new, unseen data.



Animal Classifier
Tree



How Trees Choose: Entropy and Information Gain

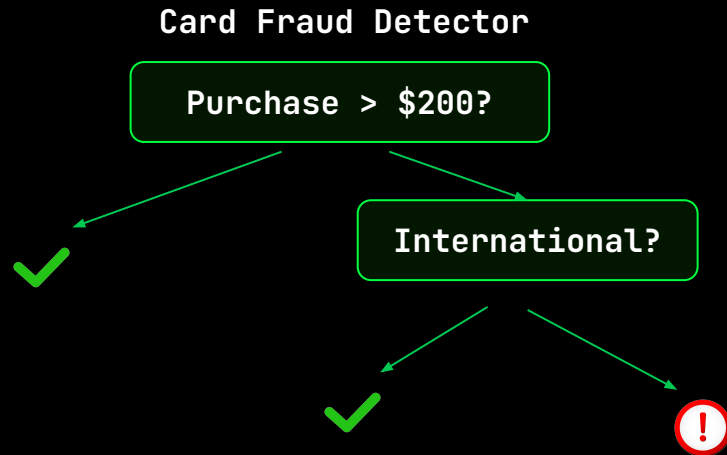
Entropy: A mathematical measure of data chaos, impurity, or disorder within a subset.

Information Gain: The net reduction in entropy achieved after splitting a dataset based on a specific feature threshold.

Optimization Goal: Decision trees choose splits that maximize Information Gain, effectively turning messy mixtures into highly pure class nodes.

Transaction_ID	Amount_Under_200	Is_International	Is_Fraud
TX_01	1	0	0
TX_02	1	1	0
TX_03	0	1	1
TX_04	0	0	0

Small example for card fraud detector



The tree chooses the 'Purchase Amount' feature first because it eliminates the most uncertainty (has highest information gain) in a single step. Splitting at the \$200 mark isolates a pure node. Splitting on International vs Domestic purchase leaves a messy 50/50 distribution.

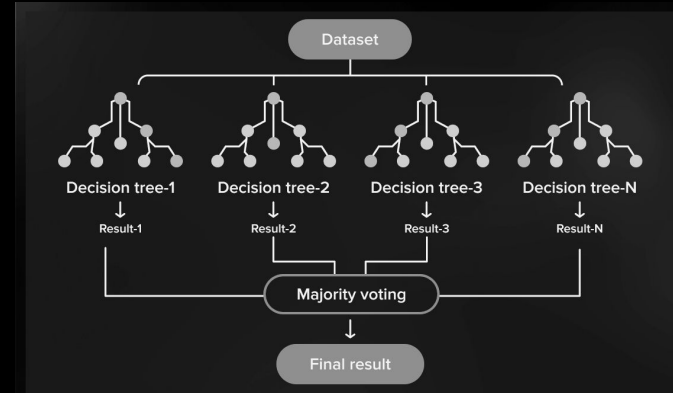
Overfitting and the Move to Ensembles

Overfitting: Occurs when an algorithm learns the background noise and random anomalies of the training dataset so perfectly that it fails to generalize to entirely new, unseen testing data.

Think: Imagine you are studying for a math test. Instead of learning the actual concepts, you just memorize the order of the multiple choice answers on the practice test (like 1. A, 2. C, 3. B).

When you take the practice test again you get a perfect score! But when you sit down for the real exam, the questions are slightly different. Because you only memorized specific answers instead of learning how to solve the problems, you end up failing.

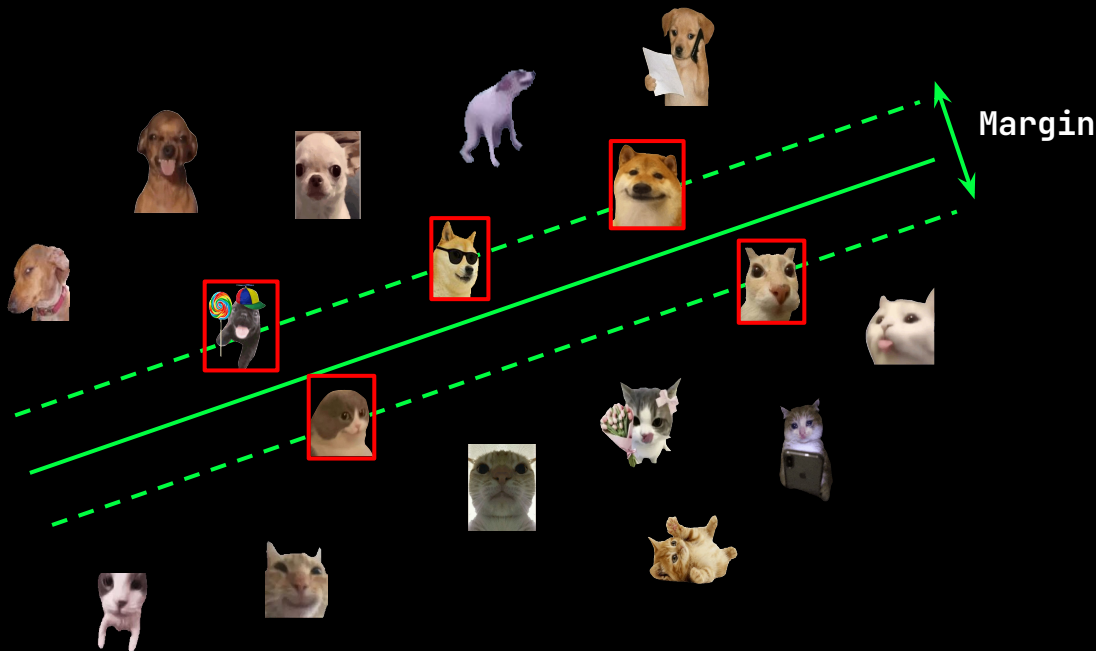
Ensembles: The strategic process of combining the predictions of multiple individual models to construct a significantly more robust global predictor



Random Forest is the technique of training hundreds of individual decision trees and letting them vote on the final classification

Support Vector Machines: Finding the Boundary

Big Idea: Constructs an optimal decision boundary (a hyperplane) in a high dimensional space to cleanly partition different classes of data.



Hard vs Soft Margin:

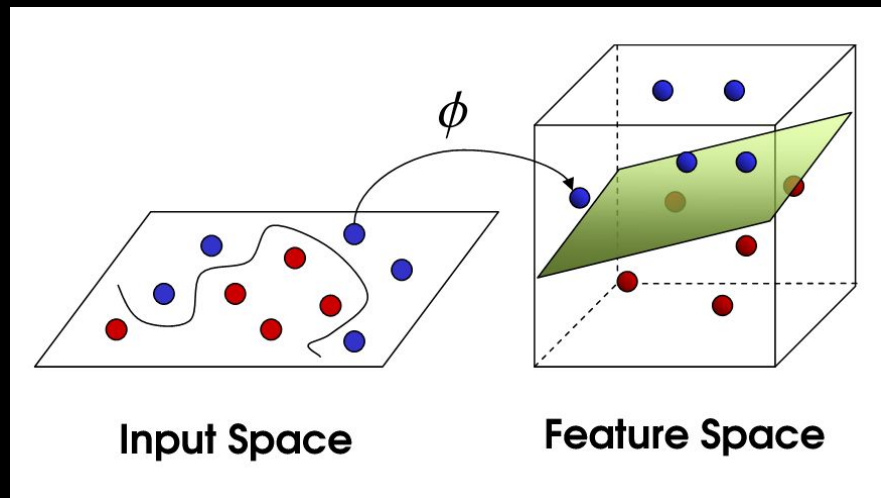
- > **Hard:** Demands perfection (no points should be within the margin).
- > **Soft:** It's ok to tolerate a few items in the neutral zone - better for generalization.

C-Parameter:

- > **High C:** Higher penalty for any data point out of place - skinnier margin.
- > **Small C:** The model cares more about generalizing to unseen data, the margin is wider.

Geometric Separability & The Kernel Trick

Real-world data is rarely perfectly separable by a straight line in its native dimension.



This concept of mapping attributes into higher-dimensional coordinate spaces to expose hidden relationships is the direct mathematical foundation of vector embeddings used in modern LLMs.

Instead of drawing highly complex, twisting curves in the original lower-dimensional space, the algorithm projects the feature coordinates into a higher-dimensional space.

Titanic Dataset Training Example (SVM)

```
from sklearn.svm import SVC

# Initialize a Soft-Margin Support Vector Classifier
# We use a standard RBF Kernel to handle potential non-linear separability
# C=1.0 balances the margin width against misclassification penalties
svm_model = SVC(kernel='rbf', C=1.0, random_state=42)

print("--- Step 1: Evaluating via Cross-Validation ---")
# 5-Fold Cross-Validation on the training set to ensure stability
cv_scores = cross_val_score(svm_model, X_train_processed, y_train, cv=5,
                             scoring='f1')
print(f"5-Fold CV F1-Scores: {cv_scores}")
print(f"Mean CV F1-Score: {cv_scores.mean():.4f}")

print("\n--- Step 2: Final Model Training ---")
# Train the model on the full processed training set
svm_model.fit(X_train_processed, y_train)
print("Model training complete.")

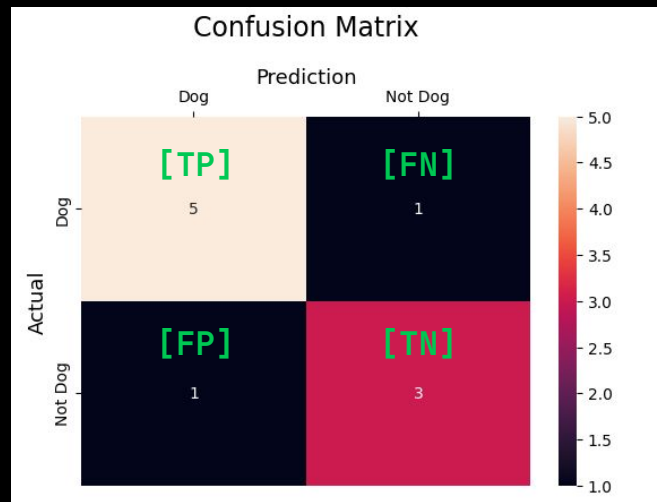
--- Step 1: Evaluating via Cross-Validation ---
5-Fold CV F1-Scores: [0.71578947 0.7311828  0.77083333 0.65853659 0.71578947]
Mean CV F1-Score: 0.7184

--- Step 2: Final Model Training ---
Model training complete.
```

How do we Know our Model is Working?

The Four Outcomes:

- > **True Positive/Negative:** Model's prediction matches reality.
- > **False Positive:** "False Alarm" error (Type 1 Error). Predicts an event that didn't happen.
- > **False Negative:** "Missed Detection" (Type 2 Error). Failing to predict an event that DID happen.



Q: Is accuracy (correct/total) enough?

Balancing Quality and Quantity: Precision vs Recall

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Core Concept:

Out of everything the model predicted positive - how many were **actually** positive?

Use Case:

Minimizing false positives. I.e. spam detection in emails needs **high precision** - we would not want to mark an important email as spam → we need to be precise.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Core Concept:

Out of all of the actual positive cases present in the data - how many did the model manage to capture?

Use Case:

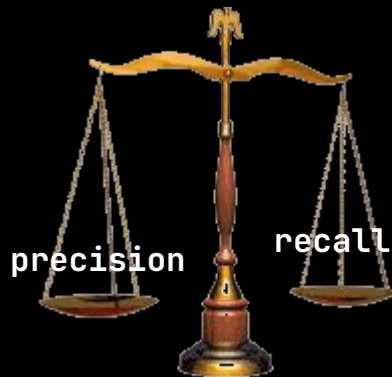
Minimizing false negatives. I.e. illness detection for patients needs **high recall** - a missed detection means the patient's illness goes undetected!

The F1 Score: A Combined Metric

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Optimizing for precision only often destroys recall and vice versa.

Think: If you make a spam filter so strict it very often classifies real emails as spam (100% Precision), then the recall will be very low (because there are a lot of false negatives).



F1 takes the Harmonic Mean of the Precision and Recall. It's more than just an average - it penalizes extreme values. If precision is 0 then F1 will be 0.

Titanic Dataset Evaluation

```

from sklearn.metrics import confusion_matrix, classification_report, f1_score

# Generate predictions on the pristine test dataset
y_pred = svm_model.predict(X_test_processed)

print("---- Step 1: The Confusion Matrix ----")
cm = confusion_matrix(y_test, y_pred)
cm_df = pd.DataFrame(cm,
                      index=['Actual Perished (0)', 'Actual Survived (1)'],
                      columns=['Predicted Perished (0)', 'Predicted Survived (1)'])

print(cm_df)

print("\n--- Step 2: Precision, Recall, & F1-Score Breakdown ---")
# This yields a complete statistical review of our classification success
print(classification_report(y_test, y_pred, target_names=['Perished',
                                                           'Survived']))

# Isolate the final F1 score target metric
final_f1 = f1_score(y_test, y_pred)
print(f"Final Test Dataset F1-Score: {final_f1:.4f}")

```

--- Step 1: The Confusion Matrix ---

	Predicted Perished (0)	Predicted Survived (1)
Actual Perished (0)	100	9
Actual Survived (1)	22	47

--- Step 2: Precision, Recall, & F1-Score Breakdown ---

	precision	recall	f1-score	support
Perished	0.82	0.92	0.87	109
Survived	0.84	0.68	0.75	69
accuracy			0.83	178
macro avg	0.83	0.80	0.81	178
weighted avg	0.83	0.83	0.82	178

Final Test Dataset F1-Score: 0.7520

Core Foundations: Complete

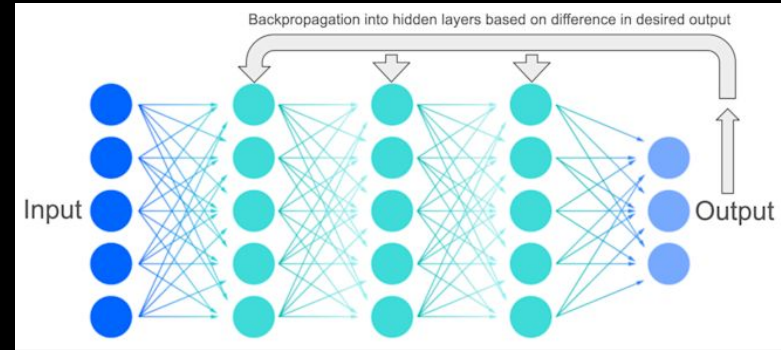
In this Crash Course we learned about:

> **Data Preprocessing:** Imputation, Feature-Engineering, One-Hot Encoding...

> **Algorithmic Mechanics + Models:** Naive Bayes, Decision Trees, SVM

> **Validation + Evaluation:** Moving past basic accuracy to Cross-Validation, Precision, Recall, F1-Score

To build an LLM, you do not throw away these foundations—you stack them millions of layers deep.



Coming up next: Ancestors of LLMS, Probabilistic Language Modeling, Transformers, and more!